Prepped — Zero-to-One Product Research Dossier

Core recommendation: Build Prepped as a **voice-first, scenario-based practice platform** with a narrow initial wedge in **English speaking fluency + job-interview readiness for Indian learners**, then expand into professional communication, negotiation, and multilingual learning. The strongest opportunity is not "another AI tutor," but a system that turns real spoken conversations into **validated, reusable practice items** and measurable skill progress.

The product should be architected around a provider-agnostic real-time voice pipeline, a scenario state machine, a learner-memory layer, and an evidence-based assessment layer. Voice AI is technically feasible today, but cost, latency, accent robustness, privacy, and assessment validity—not model availability—are the real constraints. [1] [2] [3]

1. Executive summary

Strongest product opportunity

Existing products split into three imperfect camps:

Category	Examples	Strength	Gap Prepped can exploit
Gamified language apps	Duolingo	Habit formation, broad content	Speaking practice and realistic roleplay are largely gated behind paid tiers; practice can feel game-like rather than job-realistic. [4] [5]
Pronunciation specialists	ELSA Speak	Fine-grained pronunciation feedback	Strong drills, but less emphasis on open-ended professional scenarios and long-term conversational transfer. [5] [6]
AI conversation tutors	Speak, Duolingo Max	Natural conversation and feedback	Limited transparent assessment, weak reuse of conversation mistakes into future practice, and pricing that can be high for price-sensitive learners. [7] [8] [9]

Prepped's differentiation: a closed learning loop:

1. Learner enters a realistic voice scenario.
2. AI counterpart adapts to goals, difficulty, and learner history.
3. System captures transcript, audio features, rubric evidence, and learner errors.
4. Errors become personalized review cards and future scenario constraints.
5. Progress is measured against human-anchored rubrics, not a single opaque "AI score."

This is more defensible than simply wrapping an LLM in a microphone UI.

What to build first

A **"Scenario Studio + Voice Practice Loop"**:

- Learner selects a goal: e.g., "Crack a product-manager interview," "Explain a project to an interviewer," "Speak confidently in a customer-support call."
- Prepped runs a real-time voice conversation with interruption support.
- After the session, it produces: transcript, timestamped feedback, strengths, specific improvement actions, pronunciation/fluency observations where reliable, and review cards.
- The next session deliberately reintroduces prior weaknesses.

This can start with English and Hindi-supported interfaces, then expand by language and domain.

2. User segments and jobs to be done

Primary segments

Segment	Job to be done	Willingness to pay	Initial priority
Indian college students and early-career candidates	"Help me speak confidently in placement interviews and group discussions."	Medium; price-sensitive	**High**
Working professionals	"Help me present, negotiate, and handle workplace conversations."	Higher	High after wedge
English learners	"Help me speak naturally, not just answer grammar questions."	Medium	High
Educators, bootcamps, and institutions	"Give every learner structured speaking practice and evidence of progress."	Institutional pricing	Later
Customer-facing teams	"Train agents for difficult calls with safe repetition."	High	Later

Core jobs to be done

- "Give me realistic practice before I face a real person."
- "Tell me exactly what to improve, with examples from what I actually said."
- "Make me practice my weaknesses again without making practice feel repetitive."
- "Show me that I am improving in a way I can trust."
- "Let me practice privately, safely, and without embarrassment."
- "Let me review, replay, slow down, and understand what was said."

Core user journey

```
flowchart LR
  A[Onboarding: goal, language, level, constraints] --> B[Diagnostic voice scenario]
  B --> C[Skill profile and rubric baseline]
  C --> D[Personalized scenario recommendation]
  D --> E[Real-time voice practice]
  E --> F[Transcript + audio + interaction events]
  F --> G[Structured evaluation and feedback]
  G --> H[Review cards and spaced repetition]
  H --> I[Progress dashboard]
  I --> D
  G --> J[Optional human/peer review]
  J --> I
```

Technical inference: the highest-retention loop is likely *scenario → specific feedback → targeted review → revised scenario*, rather than unlimited free chat. Free chat is useful for engagement but weak for measurable skill development.

3. Competitor and open-source landscape

Commercial products

Product	Observed positioning	Notable capabilities	Limitations / opportunity
Duolingo	Broad gamified language learning	Large course catalog; Max adds AI Roleplay and explanation features. [4] [5]	Less depth for professional scenarios; AI features vary by course and are paid-tier features. [5] [10]
Speak	Speaking-first AI English tutor	Open conversation, tutor explanations, roleplay, feedback on fluency/grammar/vocabulary. [7] [8]	Premium-oriented; less transparent public methodology for skill measurement. [8] [9]
ELSA Speak	Pronunciation and accent training	Sound-level pronunciation feedback, Speech Analyzer, AI roleplay. [5] [6]	Drill-centric; may under-serve open-ended professional communication. [5]
Duolingo Max	AI features over gamified core	Roleplay, Video Call with AI character, Explain My Answer. [11] [12]	Reported around $30/month in some markets; creates an affordability gap. [8] [12]
Generic AI chat apps	Flexible conversation	Broad topic coverage	Usually text-first, weak stateful pedagogy, weak voice UX, no validated learning model.

Vendor/product claims caveat: pricing and feature availability change frequently by country and platform. The figures above are market reports and product pages, not a guarantee of current local pricing; Prepped should verify pricing per launch market before positioning.

Open-source and infrastructure projects

Project	Role in Prepped	License / status evidence	Assessment
Pipecat	Open-source Python framework for real-time voice and multimodal AI agents. [13]	Repository identifies it as open source; exact license must be verified in the repository before commercial embedding.	Strong candidate for the voice orchestration layer.
LiveKit Agents	Framework for adding Python/Node programs to LiveKit rooms as real-time participants. [14]	Documentation describes the framework as fully open source and community-supported. [14]	Strong candidate for WebRTC rooms, turn-taking, and real-time media transport.
FSRS / Open Spaced Repetition	Scheduling review items based on recall probability, stability, and difficulty. [15] [16]	`fsrs.js` is MIT-licensed; `py-fsrs` is MIT-licensed; `fsrs-browser` is BSD-3-Clause. [17] [18] [19]	Excellent fit for conversation-derived review scheduling.
Anki/AnkiDroid	Reference product for spaced repetition and active recall.	Prepped should not clone Anki; it should adopt the underlying learning principles and interoperable export where useful.	Useful design benchmark, not necessarily a direct dependency.

Verified fact: FSRS models retrievability, stability, and difficulty, which makes it more suitable than a naive fixed-interval scheduler for personalized review. [15]

Technical inference: Prepped should use FSRS as a scheduling service, but define its own "memory objects": mispronunciations, grammar patterns, missing vocabulary, weak interview answers, hesitation patterns, and scenario-specific strategies.

4. Feature inventory and priorities

P0 — Foundation

Feature	Why it matters	Dependencies
Account, consent, privacy controls	Required before storing voice and transcripts	Auth, database, legal policy
Browser/mobile microphone capture	Core input path	WebRTC or MediaRecorder, permissions
Real-time voice conversation	Core product promise	STT, LLM, TTS, turn detection
Live captions and transcript	Accessibility, review, trust	STT, frontend state
Scenario library	Makes practice purposeful	Content model, prompt design
Post-session feedback	Converts conversation into learning	Evaluation rubrics, LLM
Basic progress dashboard	Motivation and retention	Event store, analytics
Replay with speed control	Reflection and accessibility	Audio storage, player UI
Review cards	Long-term retention	FSRS, item generation

P1 — Differentiation

- Adaptive difficulty: scenario parameters change based on prior performance.
- Branching scenario state machines: interviewer follows up, customer escalates, negotiation counterpart pushes back.
- Learner memory: remembers prior mistakes, goals, role, target companies, and preferred language.
- Pronunciation assistance: word- or phoneme-level feedback only where the speech provider's evidence is reliable.
- Shadowing mode: learner repeats model utterances with adjustable speed.
- Human review queue: optional expert or peer evaluation for high-stakes practice.
- Export/import: Anki-compatible exports, CSV, transcript export, and LMS-friendly reporting.

P2 — Advanced

- Animated avatars and visual environments.
- Multilingual roleplay and code-switching support.
- Group practice and collaborative roleplay.
- Instructor dashboards, cohort analytics, assignments, and rubric customization.
- Institutional SSO, audit logs, data residency options, and admin controls.
- Offline review for downloaded cards.
- On-device or self-hosted speech models for privacy-sensitive deployments.

Important product decision: do **not** make avatars a P0 requirement. They increase cost, latency, moderation surface, and development complexity without necessarily improving learning outcomes. A clean voice-plus-transcript interface is likely to validate the core loop faster.

5. Learning science and assessment design

Learning principles to implement

- **Active recall:** learners must produce spoken answers, not merely recognize correct options.
- **Spaced repetition:** schedule review based on predicted recall, not calendar time alone.
- **Desirable difficulty:** gradually increase scenario pressure, ambiguity, interruption, time constraints, and counterparty assertiveness.
- **Immediate, specific feedback:** feedback should quote the learner's actual utterance and offer a better alternative.
- **Transfer:** reuse learned phrases and strategies in new but related scenarios.

- **Motivation:** streaks and points can help habit formation, but should not replace evidence of competence.

Why "AI score" is not automatically valid

An LLM-generated number such as "Communication: 78/100" is **not an objectively valid measurement** unless it is tied to a defined construct, consistent rubric, calibration data, and human judgment.

A defensible assessment system should include:

1. **Construct definitions:** fluency, coherence, task completion, lexical range, grammatical accuracy, pronunciation intelligibility, professional tone, active listening, negotiation strategy.
2. **Behavioral rubrics:** each level describes observable evidence, e.g., "responds to the counterparty's objection with a relevant clarification and a concrete next step."
3. **Evidence extraction:** transcript, timestamps, interruptions, response latency, filler-word rate, vocabulary diversity, task milestones, and audio-derived features.
4. **Model-as-evaluator:** LLM classifies evidence against rubric criteria and provides rationales.
5. **Human anchoring:** experts label a calibration set; model outputs are compared with human ratings.
6. **Reliability checks:** measure inter-rater agreement, test-retest stability, and score drift across model versions.
7. **Transparent uncertainty:** show "developing / proficient / advanced" plus evidence, rather than pretending decimal precision.

Academic finding: FSRS explicitly tracks recall probability, stability, and difficulty, providing a more principled basis for review scheduling than fixed intervals. [15]

Untested hypothesis: conversation-derived review cards will improve transfer better than generic vocabulary decks. This requires an A/B experiment measuring delayed performance in a new scenario.

6. Voice and AI technology comparison

Real-time voice pipeline options

Approach	Pros	Cons	Best use
Managed end-to-end realtime voice API	Lowest integration complexity; speech, reasoning, and voice in one service	Higher per-minute/token cost; provider coupling; less control over pipeline	Fast launch, premium tier
Modular managed stack: STT + LLM + TTS	Best model choice per task; easier to swap vendors	More orchestration work; latency must be managed	Recommended default

Approach	Pros	Cons	Best use
Self-hosted open models	Data control, potentially lower marginal inference cost at scale	GPU operations, model maintenance, accent/language quality risk, high fixed cost	Privacy-sensitive or very large scale
Hybrid	Managed for launch; self-host selected components later	Operational complexity	Recommended long-term path

Verified cost signals

- OpenAI's GPT-Realtime-2.1 is listed at $32 per 1M audio input tokens and $64 per 1M audio output tokens; its mini variant is listed at $10/$20 per 1M audio input/output tokens. [20] [1]
- OpenAI also lists a $0.05 per minute / $0.00083 per second price point in its pricing material. [20]
- Deepgram lists Flux English conversational speech recognition at $0.0065/min pay-as-you-go and $0.0057/min on its Growth plan; its multilingual model is listed at $0.0078/min pay-as-you-go. [2]
- ElevenLabs' free plan is described as including 10,000 credits per month, approximately 10 minutes of audio; paid plans begin at $6/month for 30,000 credits. [21]

Cost inference: a modular pipeline using a low-cost STT service, a text LLM, and a separate TTS service can be materially cheaper than a premium end-to-end realtime model, but it requires careful latency engineering. A 10-minute daily voice session can become expensive at scale if every second uses a premium realtime model.

Required voice capabilities

- Streaming microphone capture.
- Voice activity detection and endpointing.
- Barge-in / interruption handling.
- Streaming STT with partial hypotheses.
- Streaming LLM response generation.
- Streaming TTS.
- Turn-taking policy.
- Fallback when a provider fails.
- Audio replay and caption synchronization.
- Multilingual recognition and speech generation.

Technical inference: Prepped should implement a **turn-taking controller** separate from the LLM. The controller decides when the AI may speak, when to wait, when to interrupt, and when to ask for clarification. This prevents the common failure mode where the AI talks over the learner or reacts to partial sentences.

7. Candidate architecture

Recommended stack

Layer	Primary recommendation	Alternatives	Rationale
Web frontend	Next.js + TypeScript + React	SvelteKit, Flutter	Strong ecosystem for realtime UI, auth, and PWA-style delivery
Mobile	React Native or Expo initially; native later if needed	Flutter	Reuses web product logic and team skills
Real-time transport	WebRTC via LiveKit	Raw WebRTC, WebSocket audio	LiveKit Agents is designed for realtime participants in rooms. [14]
Voice orchestration	Pipecat	Custom async pipeline	Purpose-built for realtime voice and multimodal agents. [13]
Backend API	FastAPI + Python	Node.js/NestJS	Python aligns with AI/ML tooling and Pipecat
Primary database	PostgreSQL	MySQL	Relational integrity, JSONB, migrations, analytics
Cache/queue	Redis + Celery/Arq	RabbitMQ, Kafka later	Session state, rate limits, background jobs
Object storage	S3-compatible storage	Cloudflare R2, MinIO	Audio, exports, avatars
Vector/search store	pgvector initially	Qdrant, Weaviate	Reduces infrastructure sprawl
Auth	Auth.js/Ory/Kratos or managed auth	Clerk, Supabase Auth	Must support deletion, MFA, and role-based access
Observability	OpenTelemetry + Grafana/Prometheus + Loki	Datadog	Vendor-neutral and cost-controlled
CI/CD	GitHub Actions	GitLab CI	Standard developer workflow

Architecture diagram

```
flowchart TB
  subgraph Client
    UI[Web / Mobile UI]
    Mic[Microphone capture]
    Player[Audio player and captions]
  end

  subgraph Edge
    RTC[WebRTC / LiveKit SFU]
  end

  subgraph VoiceRuntime
    VAD[Voice activity detection]
    STT[Streaming speech-to-text]
    Turn[Turn-taking controller]
    LLM[Scenario LLM]
```

```
    TTS[Streaming text-to-speech]
    Fallback[Fallback provider router]
  end

  subgraph Core
    API[Backend API]
    Scenario[Scenario engine]
    Eval[Evaluation service]
    Memory[Learner memory]
    SRS[FSRS review scheduler]
    Analytics[Progress analytics]
  end

  subgraph Data
    PG[(PostgreSQL)]
    Redis[(Redis)]
    Blob[(Object storage)]
    Vec[(pgvector)]
  end

  UI --> Mic --> RTC
  RTC --> VAD --> STT --> Turn --> LLM --> TTS --> RTC --> Player
  Turn --> Fallback --> LLM
  RTC --> API
  API --> Scenario
  Scenario --> LLM
  API --> Eval
  Eval --> Memory
  Memory --> SRS
  Eval --> Analytics
  API --> PG
  API --> Redis
  API --> Blob
  Memory --> Vec
  SRS --> PG
```

Component responsibilities

- **Client:** captures audio, renders captions, manages permissions, displays feedback, and handles reconnects.
- **RTC layer:** transports audio with low latency and supports room-based sessions.
- **Voice runtime:** handles VAD, STT, interruption, LLM calls, TTS, and provider fallback.
- **Scenario engine:** owns the scenario definition, persona, goals, milestones, difficulty, and state transitions.
- **Evaluation service:** extracts evidence and produces rubric-based feedback.
- **Memory service:** stores durable learner traits, recurring errors, mastered skills, and privacy-scoped context.
- **Review scheduler:** uses FSRS to decide when review items return.
- **Analytics service:** computes progress, engagement, funnel metrics, and quality indicators.

8. Scenario state machine

A scenario should not be a single prompt. It should be a structured, testable object.

```
{
  "scenario_id": "pm_interview_v1",
  "title": "Product Manager Interview",
  "language": "en-IN",
  "learner_role": "candidate",
  "ai_role": "interviewer",
  "objectives": [
    "Explain a project using situation-action-result structure",
    "Handle a follow-up challenge without becoming defensive",
    "Ask at least one clarifying question"
  ],
  "difficulty": {
    "initial": 2,
    "adaptive": true,
    "max": 5
  },
  "persona": {
    "name": "Aditi",
    "style": "professional, concise, probing",
    "constraints": [
      "Do not reveal rubric details",
      "Stay in interviewer role",
      "Ask one question at a time"
    ]
  },
  "states": [
    "introduction",
    "project_deep_dive",
    "challenge",
    "metrics_discussion",
    "candidate_questions",
    "closing"
  ],
  "exit_conditions": {
    "success": "candidate completes all objectives",
    "retry": "candidate requests retry or evaluation identifies major gaps"
  }
}
```

State transitions

```
stateDiagram-v2
  [*] --> Introduction
  Introduction --> ProjectDeepDive: candidate introduces experience
  ProjectDeepDive --> Challenge: interviewer probes decision
  Challenge --> MetricsDiscussion: candidate responds adequately
  Challenge --> ProjectDeepDive: candidate needs scaffolding
  MetricsDiscussion --> CandidateQuestions
```

```
  CandidateQuestions --> Closing
  Closing --> [*]
```

Technical inference: the scenario engine should distinguish **persona instructions** from **evaluation instructions**. The conversational agent should not see the full scoring rubric while roleplaying, because that can make it behave like an examiner rather than a realistic counterpart.

9. Data model

Core entities

```
erDiagram
  USER ||--o{ LEARNER_PROFILE : has
  USER ||--o{ SESSION : attends
  SCENARIO ||--o{ SESSION : used_in
  SESSION ||--o{ TURN : contains
  SESSION ||--o{ EVALUATION : produces
  EVALUATION ||--o{ EVIDENCE : contains
  EVALUATION ||--o{ REVIEW_ITEM : creates
  USER ||--o{ REVIEW_ITEM : owns
  REVIEW_ITEM ||--o{ REVIEW_LOG : has
  SCENARIO ||--o{ SCENARIO_VERSION : versioned_as
  ORGANIZATION ||--o{ USER : manages
```

Example PostgreSQL schema

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  deleted_at TIMESTAMPTZ
);

CREATE TABLE learner_profiles (
  user_id UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  target_language TEXT NOT NULL,
  interface_language TEXT NOT NULL DEFAULT 'en',
  goals JSONB NOT NULL DEFAULT '[]',
  skill_snapshot JSONB NOT NULL DEFAULT '{}',
  consent_version TEXT NOT NULL,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE scenarios (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug TEXT UNIQUE NOT NULL,
  title TEXT NOT NULL,
  domain TEXT NOT NULL,
```

```
  language TEXT NOT NULL,
  difficulty_min SMALLINT NOT NULL,
  difficulty_max SMALLINT NOT NULL,
  is_public BOOLEAN NOT NULL DEFAULT false,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE scenario_versions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  scenario_id UUID NOT NULL REFERENCES scenarios(id) ON DELETE CASCADE,
  version INTEGER NOT NULL,
  definition JSONB NOT NULL,
  prompt_hash TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (scenario_id, version)
);

CREATE TABLE sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  scenario_version_id UUID NOT NULL REFERENCES scenario_versions(id),
  started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  ended_at TIMESTAMPTZ,
  status TEXT NOT NULL DEFAULT 'active',
  audio_uri TEXT,
  transcript_uri TEXT,
  metadata JSONB NOT NULL DEFAULT '{}'
);

CREATE TABLE turns (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
  speaker TEXT NOT NULL CHECK (speaker IN ('user','agent','system')),
  started_ms INTEGER NOT NULL,
  ended_ms INTEGER,
  text TEXT,
  audio_uri TEXT,
  stt_confidence REAL,
  interrupted BOOLEAN NOT NULL DEFAULT false,
  metadata JSONB NOT NULL DEFAULT '{}'
);

CREATE TABLE evaluations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
  rubric_version TEXT NOT NULL,
  overall_band TEXT,
  dimension_scores JSONB NOT NULL DEFAULT '{}',
  strengths JSONB NOT NULL DEFAULT '[]',
  improvements JSONB NOT NULL DEFAULT '[]',
  model_metadata JSONB NOT NULL DEFAULT '{}',
  human_review_status TEXT NOT NULL DEFAULT 'none',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```
CREATE TABLE review_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  session_id UUID REFERENCES sessions(id) ON DELETE SET NULL,
  type TEXT NOT NULL,
  front TEXT NOT NULL,
  back TEXT NOT NULL,
  evidence JSONB NOT NULL DEFAULT '{}',
  fsrs_state JSONB NOT NULL DEFAULT '{}',
  due_at TIMESTAMPTZ NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE review_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  review_item_id UUID NOT NULL REFERENCES review_items(id) ON DELETE CASCADE,
  reviewed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  rating SMALLINT NOT NULL,
  scheduled_interval_days REAL,
  actual_interval_days REAL
);
```

Key API contracts

```
POST /v1/scenarios/{scenario_id}/sessions
{
  "language": "en-IN",
  "difficulty": "adaptive",
  "audio_consent": true,
  "device_profile": { "platform": "web", "network": "4g" }
}
```

```
POST /v1/sessions/{session_id}/events
{
  "type": "user_turn_completed",
  "turn_id": "turn_123",
  "duration_ms": 8400,
  "interrupted_agent": false
}
```

```
POST /v1/sessions/{session_id}/evaluate
{
  "rubric_version": "communication-v1",
  "include_audio_analysis": true,
  "generate_review_items": true
}
```

```
GET /v1/reviews/due?limit=20
```

```
POST /v1/reviews/{review_item_id}/grade
{
  "rating": "good",
  "response_ms": 3200
}
```

Background jobs

- Transcribe and archive completed sessions.
- Generate evaluation after session end.
- Generate and deduplicate review items.
- Schedule FSRS reviews.
- Delete expired audio according to retention policy.
- Recompute learner skill snapshots.
- Run model-quality evaluation suites.
- Export user data on request.
- Detect abuse, prompt injection, and unsafe content.

10. End-to-end voice flow

```
sequenceDiagram
  participant U as Learner
  participant C as Client
  participant R as RTC/LiveKit
  participant V as Voice Runtime
  participant S as STT
  participant L as LLM
  participant T as TTS
  participant B as Backend
  participant E as Evaluator
  participant F as FSRS

  U->>C: Start scenario
  C->>R: Join room and request mic
  R->>V: Stream audio
  V->>S: Stream audio
  S-->>V: Partial transcript
  V->>L: Final transcript + scenario state
  L-->>V: Agent response tokens
  V->>T: Stream response text
  T-->>R: Streaming audio
  R-->>C: Agent audio + captions
  U->>V: Interrupts agent
  V->>L: Stop generation, process new user input
  U->>C: End session
  C->>B: Finalize session
  B->>E: Run rubric evaluation
```

```
    E->>B: Feedback and evidence
    B->>F: Create/update review items
    B-->>C: Feedback, progress, due reviews
```

Latency budget

Stage	Target	Notes
Microphone to STT partial	< 200 ms	Local/browser capture plus network
Endpoint decision	300–700 ms	Must balance responsiveness vs. cutting users off
LLM first token	< 500 ms	Use streaming and small scenario context
TTS first audio	< 300 ms	Streaming TTS required
Perceived response latency	< 1.2–1.8 s	Acceptable for conversational flow
Recovery after interruption	< 500 ms	Cancel in-flight generation immediately

Untested hypothesis: Indian-network conditions and mobile microphone quality may require more conservative endpointing than typical Western-desktop benchmarks. This must be tested with real users on 4G and low-end Android devices.

11. Security, privacy, and compliance

Data categories

- Account data: email, authentication identifiers.
- Learning data: goals, skill profile, review history.
- Conversation data: transcripts, audio, timestamps, evaluation results.
- Device/network data: browser, app version, connection quality.
- Optional biometric-derived speech features: only if explicitly needed and consented.

Privacy controls

- Explicit, granular consent for microphone access, audio storage, model processing, and research use.
- Default retention limits for raw audio; transcripts may have a separate, longer retention period only with consent.
- User-initiated export and deletion.
- Pseudonymization for analytics.
- Regional data storage options for institutional customers.
- No training foundation models on user audio without separate explicit consent.
- Clear disclosure that the counterpart is AI-generated.

Threat model

Threat	Mitigation
Prompt injection through user speech	Treat transcripts as untrusted data; separate system instructions; validate tool calls; constrain scenario actions
Persona jailbreak	Scenario state machine, output validators, refusal policies, red-team suite
Voice/audio abuse	Rate limits, abuse classifiers, human-review queue, account restrictions
Unauthorized access	RBAC, session tokens, encryption in transit and at rest, audit logs
Data leakage to vendors	Data-processing agreements, provider controls, optional self-hosted path
Insecure generated exercises	Schema validation, content moderation, human review for public scenarios
Model drift	Version pinning, regression suites, evaluation datasets, canary deployments

Regulatory considerations

- **EU AI Act:** entered into force on 1 August 2024 and became applicable on 2 August 2026, with certain exceptions; it includes transparency rules and obligations for general-purpose AI model providers. [22] [23]
- Providers of GPAI models must maintain technical documentation, implement a copyright policy, and publish a training-content summary; systemic-risk models face additional obligations. [24] [25]
- Synthetic-content transparency obligations under Article 50 are reported to apply from 2 August 2026, with a compliance date of 2 December 2026 for some pre-existing systems. [23] [26]
- **India DPDP Act:** Prepped should obtain purpose-limited consent, provide notice, support withdrawal and deletion, and maintain records of processing. Legal counsel should validate the final implementation for Indian deployment.

Verified fact: WCAG 2.2 is a W3C Recommendation organized around perceivable, operable, understandable, and robust principles. [3] [27]

12. Accessibility design

Prepped should target **WCAG 2.2 AA** from the beginning. [27] [3]

Required capabilities:

- Full captions for every voice interaction.
- Text-based alternative for every voice scenario.
- Adjustable speech rate and replay.
- Keyboard-operable interface.
- Screen-reader-compatible feedback and progress information.
- High-contrast and reduced-motion modes.

- Clear microphone permission states and error recovery.
- Avoid relying on color alone for feedback.
- Provide extended time or no-timer modes.
- Support assistive input where speech is not possible.

Important inference: accessibility is not only an ethical requirement; it expands the market to learners with hearing, motor, speech, and cognitive differences.

13. Reliability, testing, and evaluation

Reliability requirements

- Graceful degradation: if TTS fails, show text; if realtime STT fails, allow typed input; if LLM fails, retry with fallback provider.
- Session resume after network interruption.
- Idempotent event ingestion.
- Provider circuit breakers.
- Rate limiting per user, IP, session, and organization.
- Dead-letter queues for failed evaluation jobs.
- Daily database backups with tested restore procedures.
- Disaster-recovery runbook and RTO/RPO targets.

Test layers

Layer	Tests
Unit	FSRS scheduling, rubric parsing, scenario transitions, permission logic
Integration	Auth, session creation, transcript persistence, review generation
Contract	Provider adapters, API schemas, webhook handling
Voice E2E	Simulated audio, interruption, reconnect, latency measurement
AI quality	Golden transcripts, rubric agreement, hallucination checks, injection resistance
Load	Concurrent rooms, STT/TTS quotas, database contention
Accessibility	Automated WCAG checks plus manual screen-reader testing
Security	Dependency scanning, secrets scanning, penetration testing, abuse red-teaming

Evaluation datasets

Build four datasets:

1. **Diagnostic set:** known learner profiles and expected skill signals.
2. **Rubric calibration set:** expert-rated sessions.
3. **Adversarial set:** prompt injection, roleplay escape, offensive content, and false-confident feedback.
4. **Regression set:** representative sessions replayed after every model or prompt change.

Technical inference: model upgrades should be treated like database migrations: versioned, evaluated, canary-released, and reversible.

14. Licensing and cost analysis

License posture

Component	Recommended approach	License caution
Pipecat	Use for voice orchestration	Verify repository license and dependency licenses before commercial distribution. [13]
LiveKit Agents	Use for realtime rooms/agents	Verify server licensing and hosted-service terms. [14]
FSRS libraries	Use for scheduling	`fsrs.js` and `py-fsrs` are MIT; `fsrs-browser` is BSD-3-Clause. [17] [18] [19]
PostgreSQL, Redis, FastAPI, React	Use freely	Respect individual project licenses
Proprietary AI APIs	Commercial terms apply	Avoid sending sensitive audio without appropriate agreements

Zero-mandatory-spend development strategy

A zero-spend path is feasible for development, but not for unlimited real users.

Need	Free/low-cost option	Limitation
Code hosting	GitHub free tier	Private-repo and Actions limits
Database	Supabase/Neon free tier or local PostgreSQL	Connection, storage, and sleep limits
Auth	Self-hosted auth or free managed tier	Operational burden or feature limits
Voice development	Browser Web Speech APIs, local Whisper, local TTS	Quality, latency, accent coverage, and device variability vary
Hosting	Fly.io/Railway free tiers, local Docker, Oracle/AWS free tiers where eligible	Ephemeral instances, limited CPU/RAM, bandwidth limits
AI inference	Free tiers or local small models	Quotas, rate limits, weaker quality, GPU requirements

Need	Free/low-cost option	Limitation
Storage	Local MinIO or free object-storage tiers	Capacity and egress limits
Monitoring	Grafana/Prometheus/Loki self-hosted	Requires maintenance

Verified fact: ElevenLabs' free tier is described as approximately 10 minutes of audio per month, which is insufficient for meaningful real-user voice practice at scale.[21]

Technical inference: for a free-only public launch, Prepped should use **short sessions, strict daily quotas, caching, smaller models, and text fallback**. Otherwise, a single active user can consume more inference cost than the product can sustain.

Best unrestricted architecture

For a serious launch:

- Managed realtime voice provider for premium sessions.
- Modular STT + LLM + TTS for standard sessions.
- PostgreSQL with read replicas.
- Redis for session state and queues.
- Object storage with lifecycle policies.
- Kubernetes or managed containers once concurrent sessions justify operational overhead.
- Dedicated evaluation environment with recorded synthetic audio.
- Multiple model providers behind one internal `VoiceAgent` interface.

Cost-control levers

- Cache scenario instructions and static context.
- Use smaller models for routing, moderation, and simple feedback.
- Reserve premium realtime models for high-value scenarios.
- Limit raw-audio retention.
- Compress audio where acceptable.
- Batch offline evaluation rather than evaluating every turn in real time.
- Charge or quota by voice minutes, not only by subscription.
- Offer text-based practice as a lower-cost tier.

15. Key risks and mitigations

Risk	Impact	Mitigation
High inference cost	Unsustainable unit economics	Tiered models, quotas, caching, modular pipeline, paid voice-minute limits

Risk	Impact	Mitigation
Poor accent recognition	Learner frustration, invalid feedback	Test with Indian English, regional accents, noisy environments; allow transcript correction
AI feedback overclaims	Loss of trust	Evidence-linked feedback, confidence labels, human calibration
Latency makes conversation unnatural	Low retention	Streaming everywhere, turn-taking controller, regional deployment
Prompt injection	Persona break, unsafe output	Input/output validation, tool permissioning, red-team suite
Privacy concerns around voice	Regulatory and trust risk	Minimal retention, explicit consent, deletion, self-host option
Generic “AI tutor” positioning	Weak differentiation	Focus on scenario quality, memory, and measurable improvement
Content quality bottleneck	Inconsistent learning value	Scenario schema, editorial review, community templates with moderation
Vendor lock-in	Cost and migration risk	Provider abstraction, standardized event schema, exportable data
Overbuilding before validation	Wasted effort	Validate voice loop and feedback quality before avatars, institutions, and group features

16. Implementation roadmap

Phase 0 — Research validation: 2–4 weeks

Goal: prove demand and technical feasibility.

- Interview 20–30 target learners.
- Run Wizard-of-Oz voice sessions using human-operated AI or manual prompts.
- Test 5 interview scenarios and 5 English-speaking scenarios.
- Measure: completion rate, perceived usefulness, willingness to repeat, latency tolerance, and feedback usefulness.
- Build a small rubric-calibration set with expert ratings.

Acceptance criteria:

- At least 60% of test users complete a full scenario.
- At least 50% say they would practice again within a week.
- Feedback is rated specific and actionable by most testers.
- Latency is acceptable on real mobile networks.

Phase 1 — Voice practice MVP: 6–10 weeks

Goal: deliver the core loop.

- Auth, consent, profiles.
- 10 curated scenarios.
- Real-time voice conversation with captions and interruption.
- Session transcript and replay.
- Rubric-based post-session feedback.
- Basic progress dashboard.
- Review-card generation and FSRS scheduling.
- Provider abstraction for STT, LLM, and TTS.

Acceptance criteria:

- A learner can complete a 5–10 minute voice session on web and mobile web.
- Captions match the conversation with acceptable delay.
- Session artifacts persist reliably.
- Every feedback claim links to transcript evidence.
- Users can delete audio and account data.

Phase 2 — Adaptive learning: 8–12 weeks

- Adaptive difficulty.
- Learner memory boundaries and personalization.
- Branching scenarios.
- Pronunciation and fluency analytics where reliable.
- Shadowing and replay drills.
- Spaced repetition dashboard.
- A/B tests for feedback formats and review scheduling.
- Human review workflow.

Acceptance criteria:

- Repeat sessions demonstrably target prior weaknesses.
- Review items are generated without excessive duplication.
- Evaluation scores show acceptable agreement with expert ratings on the calibration set.
- System passes adversarial and privacy tests.

Phase 3 — Scale and institutions: 12–20 weeks

- Organization accounts, roles, and instructor tools.
- Cohort analytics and assignments.
- SSO and audit logs.
- Multilingual expansion.
- Group/collaborative practice.
- Export/import workflows.
- Regional deployment and data-residency options.
- Cost dashboards and quota management.

Acceptance criteria:

- Institutions can create, assign, and review scenarios.
- Admins can manage users, retention, and exports.
- Platform supports concurrent sessions with defined SLOs.
- Cost per active learner is measured and controlled.

Phase 4 — Advanced product surface

- Animated avatars.
- Visual environments.
- Marketplace for scenario creators.
- Certification or verifiable skill evidence.
- Deeper LMS and HR-system integrations.

17. Unresolved questions and experiments

Question	Recommended experiment	Success signal
Does voice practice improve real interview performance?	Pre/post study with mock interviews rated by humans	Significant rubric improvement and positive user reports
Which feedback format changes behavior?	A/B test: numeric scores vs. evidence-linked coaching	Higher repeat rate and improved next-session rubric outcomes
What latency is acceptable on Indian mobile networks?	Field test across 4G, 3G, and low-end Android	Session completion and interruption quality
Are conversation-derived review cards better than generic cards?	Randomized controlled study	Better delayed recall and transfer
Can AI evaluation match expert judgment?	Expert-labeled calibration dataset	Acceptable agreement and low score drift
Is pronunciation feedback reliable across accents?	Diverse accent test corpus	Low false-positive and false-negative rates

Question	Recommended experiment	Success signal
Will users accept audio retention for replay?	Consent and retention preference testing	High opt-in without trust concerns
What is sustainable cost per active learner?	Instrument token/minute usage by scenario type	Cost below target gross-margin threshold

18. Source register

- LiveKit Agents documentation: https://docs.livekit.io/agents/ [14]
- Pipecat repository: https://github.com/pipecat-ai/pipecat [13]
- OpenAI API pricing: https://developers.openai.com/api/docs/pricing [1]
- OpenAI pricing page: https://openai.com/api/pricing/ [20]
- OpenAI realtime model announcement: https://openai.com/index/advancing-voice-intelligence-with-new-models-in-the-api/ [28]
- Deepgram pricing: https://deepgram.com/pricing [2]
- ElevenLabs TTS pricing: https://elevenlabs.io/text-to-speech [21]
- FSRS JavaScript repository: https://github.com/open-spaced-repetition/fsrs.js [17]
- py-fsrs license: https://github.com/open-spaced-repetition/py-fsrs/blob/main/LICENSE [18]
- fsrs-browser license: https://github.com/open-spaced-repetition/fsrs-browser/blob/main/LICENSE [19]
- Open Spaced Repetition organization: https://github.com/open-spaced-repetition [29]
- WCAG 2.2 Recommendation: https://www.w3.org/TR/WCAG22/ [3]
- WCAG overview: https://www.w3.org/WAI/standards-guidelines/wcag/ [27]
- EU AI Act policy page: https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai [22]
- EU GPAI provider guidance: https://digital-strategy.ec.europa.eu/en/policies/guidelines-gpai-providers [30]
- EU GPAI obligations: https://digital-strategy.ec.europa.eu/en/factpages/general-purpose-ai-obligations-under-ai-act [25]
- Duolingo/Speak/ELSA market comparison: https://dupple.com/learn/best-ai-language-learning-tools [7]
- Duolingo Max and competitor comparison: https://www.borderset.com/blogs/posts/top-10-ai-language-learning-apps-2026 [4]
- ELSA and Duolingo comparison: https://getfluently.app/blog/best-english-learning-apps [5]
- Speak/ELSA/Duolingo pricing comparison: https://www.aitoolbox.hk/articles/best-ai-language-learning-2026-duolingo-max-vs-speak-vs-elsa-vs-memrise/ [8]
- FSRS explainer: https://research.yuda.me/podcast/episodes/algorithms-for-life/ep1-spaced-repetition/report.html [15]

⁂

1. https://developers.openai.com/api/docs/pricing
2. https://deepgram.com/pricing
3. https://www.w3.org/TR/WCAG22/
4. https://www.borderset.com/blogs/posts/top-10-ai-language-learning-apps-2026
5. https://getfluently.app/blog/best-english-learning-apps
6. https://www.talkdrill.com/blog/compare/best-ai-english-speaking-apps-2026/
7. https://dupple.com/learn/best-ai-language-learning-tools
8. https://www.aitoolbox.hk/articles/best-ai-language-learning-tools-2026-duolingo-max-vs-speak-vs-elsa-vs-memrise/
9. https://resources.rework.com/tools/ai-tools/best-ai-tools-for-language-learning-2026
10. https://speakshark.com/blog/best-free-apps-english-speaking-practice-2026
11. https://fast.io/resources/best-ai-for-language-learning-2026/
12. https://languatalk.com/blog/whats-the-best-ai-for-language-learning/
13. https://github.com/pipecat-ai/pipecat
14. https://docs.livekit.io/agents/
15. https://research.yuda.me/podcast/episodes/algorithms-for-life/ep1-spaced-repetition/report.html
16. https://github.com/open-spaced-repetition/fsrs-rs
17. https://github.com/open-spaced-repetition/fsrs.js/
18. https://github.com/open-spaced-repetition/py-fsrs/blob/main/LICENSE
19. https://github.com/open-spaced-repetition/fsrs-browser/blob/main/LICENSE
20. https://openai.com/api/pricing/
21. https://elevenlabs.io/text-to-speech
22. https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai
23. https://ai-act-service-desk.ec.europa.eu/en/ai-act/faq/when-does-enforcement-start
24. https://interoperable-europe.ec.europa.eu/collection/ai-public-sector/legal-and-procurement-guidance
25. https://digital-strategy.ec.europa.eu/en/factpages/general-purpose-ai-obligations-under-ai-act
26. https://artificialintelligenceact.eu/implementation-timeline/
27. https://www.w3.org/WAI/standards-guidelines/wcag/
28. https://openai.com/index/advancing-voice-intelligence-with-new-models-in-the-api/
29. https://github.com/open-spaced-repetition
30. https://digital-strategy.ec.europa.eu/en/policies/guidelines-gpai-providers
31. https://perplexityaimagazine.com/ai-tools/best-ai-for-language-learning/
32. https://en.ai-pedias.com/blog/ai-english-learning-2026
33. https://beconfident.app/en-us/cms/pt-br/melhores-apps-ia-aprender-ingles
34. https://accessibility.education.gov.uk/guidelines/wcag
35. https://www.gov.uk/service-manual/helping-people-to-use-your-service/understanding-wcag

36. https://www.w3.org/WAI/WCAG22/Understanding/

37. https://www.w3.org/TR/wcag/all/

38. https://www.w3.org/WAI/standards-guidelines/wcag/new-in-22/

39. https://w3c.github.io/wcag/understanding/intro

40. https://developer.mozilla.org/en-US/docs/Web/Accessibility/Guides/Understanding_WCAG

41. https://w3c.github.io/wcag/requirements/22/

42. https://ai-act-service-desk.ec.europa.eu/en/faq

43. https://digital-strategy.ec.europa.eu/en/policies/ai-code-practice

44. https://artificialintelligenceact.eu/high-level-summary/

45. https://artificialintelligenceact.eu/enforcement-of-chapter-v-under-the-eu-ai-act/